

MULTI-STAGE CI/CD PIPELINE (CORE JENKINSFILE PROJECT)

1. Definition

This project demonstrates the implementation of a **Continuous Integration and Continuous Deployment (CI/CD) pipeline** using **GitHub, Jenkins, Docker, DockerHub, and AWS EC2**. The goal of the project is to automate the process of building, packaging, and deploying a web application.

Whenever a developer pushes code to GitHub, Jenkins automatically triggers a pipeline that builds a Docker image of the application, pushes the image to DockerHub, and deploys the application container on an AWS EC2 server.

This pipeline reduces manual deployment effort and ensures consistent and reliable application delivery.

2. Scope of the Project

The scope of this project includes:

- Hosting source code in GitHub
- Automating build and deployment using Jenkins
- Containerizing the application using Docker
- Storing Docker images in DockerHub
- Deploying containers on AWS EC2
- Providing automated CI/CD workflow from code commit to deployment

This project demonstrates how DevOps tools work together to automate application delivery pipelines.

3. Methodologies

The project follows modern **DevOps CI/CD methodologies**.

Continuous Integration (CI)

Developers push code changes to GitHub. Jenkins automatically detects the change and starts the build pipeline.

Containerization

The application is packaged as a Docker image to ensure consistency across environments.

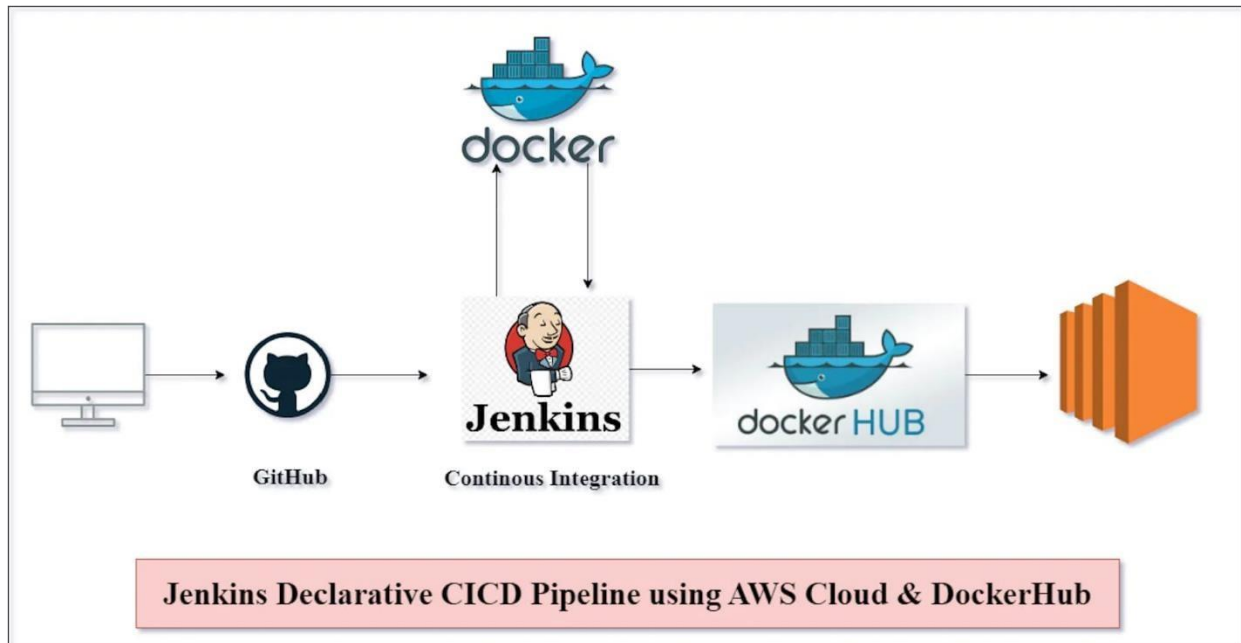
Continuous Deployment (CD)

After building the Docker image, Jenkins pushes the image to DockerHub and deploys the latest container on the EC2 instance.

Automation

All build and deployment steps are automated using a **Jenkins Declarative Pipeline defined in a Jenkinsfile**.

4. Architecture Diagram / Flow



Pipeline Flow:

Developer → GitHub → Jenkins Pipeline → Docker Build → DockerHub → EC2 Deployment → User Access

5. Explanation of Services Used

GitHub

Purpose: GitHub stores the application source code and triggers the Jenkins pipeline whenever code changes are pushed.

Jenkins

Purpose: Jenkins is an automation server used to implement the CI/CD pipeline. It pulls code from GitHub, builds Docker images, pushes them to DockerHub, and deploys the container.

Docker

Purpose: Docker is used to containerize the application, ensuring the application runs the same way across different environments.

DockerHub

Purpose: DockerHub acts as a cloud registry that stores Docker images built by Jenkins.

AWS EC2

Purpose: EC2 provides a cloud virtual machine where Jenkins runs and where the Docker containerized application is deployed.

Jenkinsfile

Purpose: Defines the pipeline stages and automates the CI/CD workflow.

6. Jenkins Pipeline Stages

The Jenkins Declarative Pipeline contains the following stages:

Stage 1 – Checkout Source Code

Jenkins pulls the application source code from the GitHub repository.

Example:

```
git clone <github-repository-url>
```

Purpose: Ensures Jenkins always builds the latest code.

Stage 2 – Build Docker Image

Jenkins builds the Docker image using the Dockerfile present in the project.

```
docker build -t username/onlinebookstore:${BUILD_NUMBER} .
```

Purpose: Converts application code into a container image.

Stage 3 – Login to DockerHub

Jenkins authenticates to DockerHub using stored credentials.

```
docker login -u USERNAME -p PASSWORD
```

Purpose: Allows Jenkins to push images to DockerHub.

Stage 4 – Push Image to DockerHub

The built Docker image is pushed to DockerHub.

```
docker push username/onlinebookstore:${BUILD_NUMBER}
```

Purpose: Stores the image so it can be pulled during deployment.

Stage 5 – Deploy Container

Jenkins deploys the application container on the EC2 instance.

Stop existing container:

```
docker stop bookstore || true
```

Remove old container:

```
docker rm bookstore || true
```

Run new container:

```
docker run -d -p 80:8080 --name bookstore username/onlinebookstore:${BUILD_NUMBER}
```

Purpose: Deploys the updated application automatically.

7. Step-by-Step Implementation

Step 1 – Launch EC2 Instance

Create an AWS EC2 instance and open the following ports:

22 – SSH access 8080 – Jenkins 80 – Application access

Step 2 – Install Docker

```
sudo yum update -y
```

```
sudo yum install docker -y
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

Verify installation:

```
docker --version
```

Step 3 – Install Jenkins

Install Java:

```
sudo yum install java-17-amazon-corretto -y
```

Install Jenkins:

```
sudo yum install jenkins -y
```

Start Jenkins:

```
sudo systemctl start jenkins
```

```
sudo systemctl enable jenkins
```

Access Jenkins:

http://EC2_PUBLIC_IP:8080

Step 4 – Configure Jenkins

Install plugins:

- Docker Pipeline • Git • Pipeline

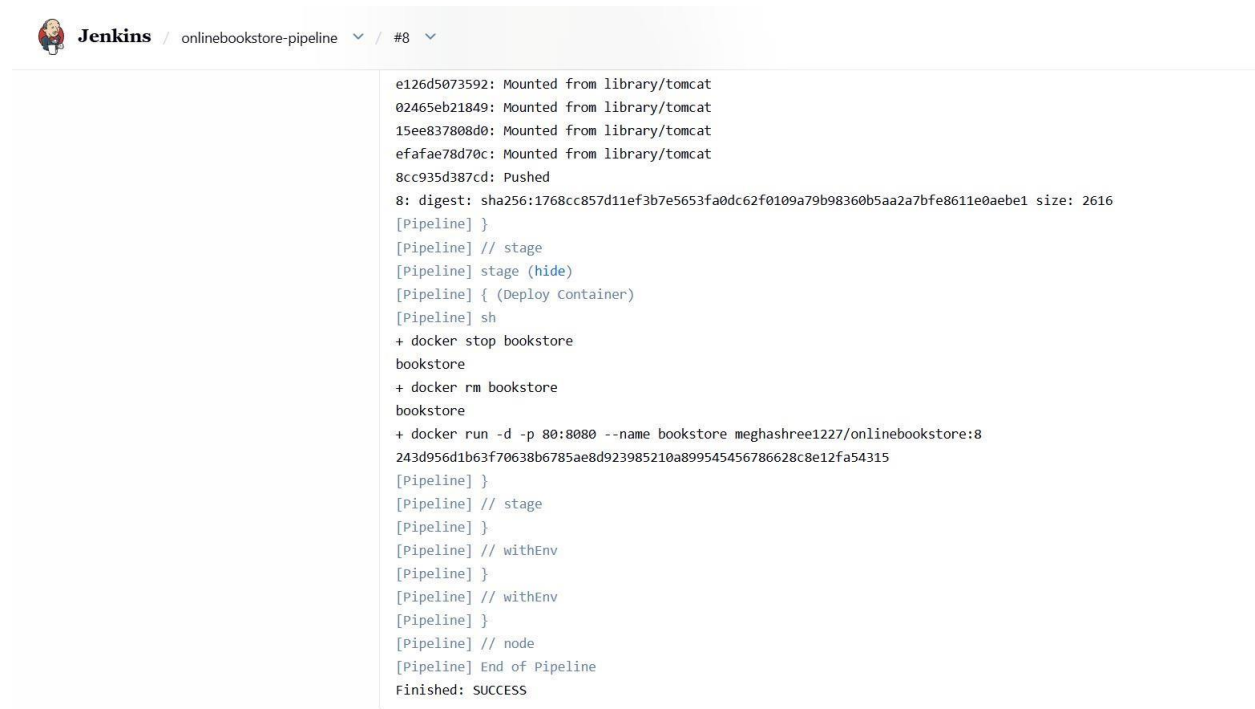
Add credentials for:

- GitHub • DockerHub

Step 5 – Create Jenkins Pipeline

Create a **Pipeline Job** in Jenkins and configure the GitHub repository.

Add Jenkinsfile containing pipeline stages.



The screenshot shows the Jenkins web interface. At the top, there's a header with the Jenkins logo, the text "Jenkins", and a breadcrumb trail: "onlinebookstore-pipeline" followed by a dropdown arrow, and "#8" followed by a dropdown arrow. Below the header, the console output for a pipeline job is displayed. The output shows various stages and steps, including mounting volumes, pushing to a repository, and running a Docker container. The final status is "Finished: SUCCESS".

```
e126d5073592: Mounted from library/tomcat
02465eb21849: Mounted from library/tomcat
15ee837808d0: Mounted from library/tomcat
efafae78d70c: Mounted from library/tomcat
8cc935d387cd: Pushed
8: digest: sha256:1768cc857d11ef3b7e5653fa0dc62f0109a79b98360b5aa2a7bfe8611e0aeb1 size: 2616
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage (hide)
[Pipeline] { (Deploy Container)
[Pipeline] sh
+ docker stop bookstore
bookstore
+ docker rm bookstore
bookstore
+ docker run -d -p 80:8080 --name bookstore meghashree1227/onlinebookstore:8
243d956d1b63f70638b6785ae8d923985210a899545456786628c8e12fa54315
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Step 6 – Run Pipeline

Trigger the pipeline manually or by GitHub webhook.

Jenkins will:

1. Pull code
2. Build Docker image
3. Push image to DockerHub
4. Deploy container on EC2

```
[root@ip-172-31-18-204 ec2-user]# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
243d956d1b63   meghashree1227/onlinebookstore:8   "catalina.sh run"       3 minutes ago Up 3 minutes  0.0.0.0:80->8080/tcp, :::80->8080/tcp
d2f28f7940da   jenkins/jenkins:lts                "/usr/bin/tini -- /u..." 11 hours ago  Up 29 minutes  0.0.0.0:8080->8080/tcp, :::8080->8080/tcp, 0.0.0.0:50000->50000/tcp, :::50000->50000/tcp
[root@ip-172-31-18-204 ec2-user]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
meghashree1227/onlinebookstore   8           f2de53a32b2c  3 minutes ago  423MB
meghashree1227/onlinebookstore   6           6a2eadf7ccf8  27 minutes ago  335MB
meghashree1227/onlinebookstore   7           6a2eadf7ccf8  27 minutes ago  335MB
jenkins/jenkins                lts         6c9cebd3534e  7 days ago     486MB
[root@ip-172-31-18-204 ec2-user]#
```

anushas76761/onlinebookstore

Last pushed 5 days ago · Repository size: 524.6 MB · ☆0 · ↓52

Add a description ⓘ ⓘ

Add a category ⓘ ⓘ

General Tags Image Management Collaborators Webhooks Settings

Tags

This repository contains 4 tag(s).

Tag	OS	Type	Pulled	Pushed
8	linux	Image	5 days	5 days
7	linux	Image	5 days	5 days
6	linux	Image	5 days	5 days
latest	linux	Image	3 days	6 days

See all

Docker commands

To push a new tag to this repository:

```
docker push meghashree1227/onlinebookstore: tagname
```

buildcloud

Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

Go to Docker Build Cloud →

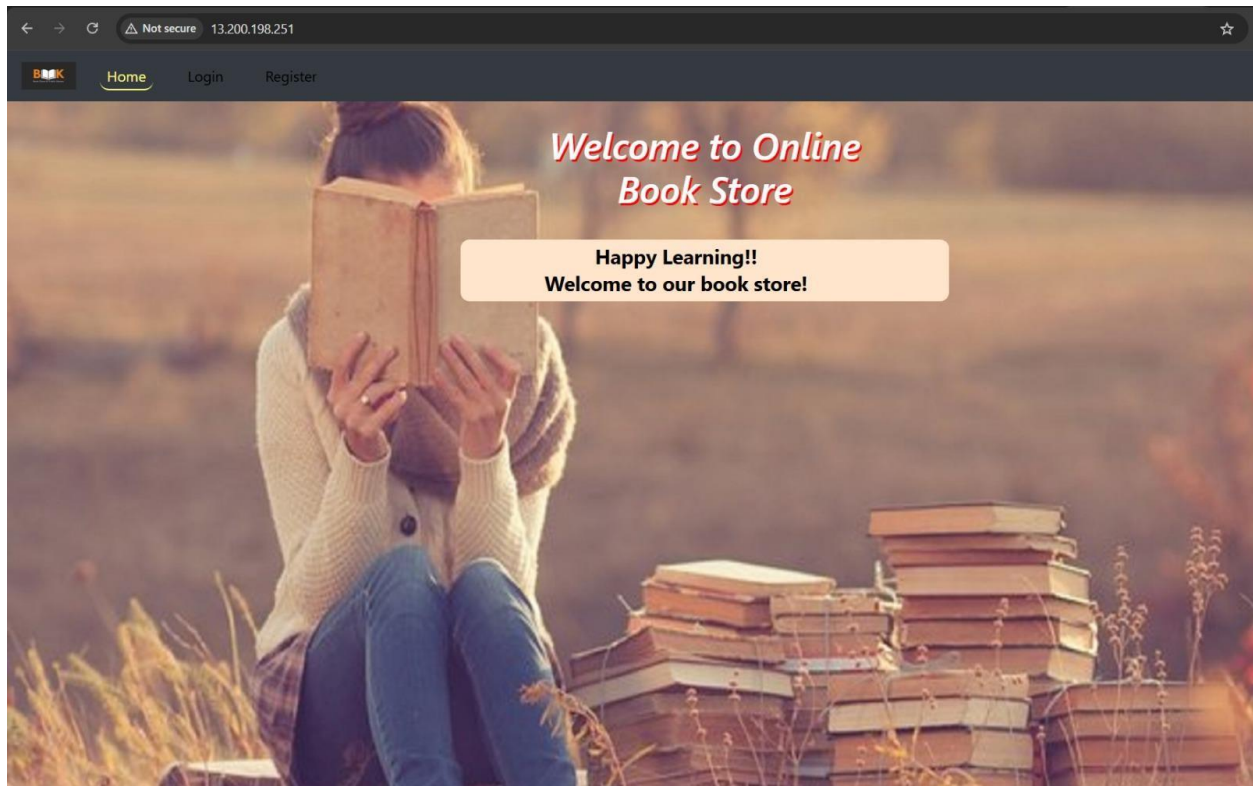
Step 7 – Access Application

Open the browser using EC2 Public IP:

http://EC2_PUBLIC_IP

Example:

The Online Bookstore application will load successfully.



8. Troubleshooting

Issue 1 – Jenkins pipeline failed

Method: Check Jenkins Console Output.

Solution: Fix syntax errors in Jenkinsfile or Docker commands.

Issue 2 – Docker image not pushed

Method: Verify DockerHub credentials.

Solution: Update Jenkins credentials configuration.

Issue 3 – Container not running

Method:

`docker ps`

Solution: Check container logs:

`docker logs <container-id>`

Issue 4 – Website not opening

Method: Verify port mapping.

Solution: Ensure container runs with:

```
docker run -d -p 80:8080
```

Also confirm AWS Security Group allows port 80.

9. Conclusion

This project successfully demonstrates a **Jenkins Declarative CI/CD pipeline using AWS EC2, Docker, and DockerHub**. The pipeline automates application build and deployment whenever new code is pushed to GitHub.

By integrating GitHub, Jenkins, Docker, and DockerHub, the system enables faster, reliable, and automated software delivery. This project represents a real-world DevOps workflow used by organizations to streamline application deployment in cloud environments.